

CKA - Core Concepts 섹션 핵심 정리

1. Cluster Architecture

쿠버네티스 클러스터는 크게 **Control Plane(마스터)** 과 **Worker Node** 로 구성됨.

Control Plane (마스터 노드)

└── kube-apiserver	# 모든 요청의 관문
└── etcd	# 클러스터 상태 저장소 (DB)
└── kube-scheduler	# Pod를 어느 노드에 배치할지 결정
└── kube-controller-manager	# 클러스터 상태를 원하는 상태로 유지

Worker Node (여러 대)

└── kubelet	# 각 노드에서 Pod 실행/관리 담당
└── kube-proxy	# 네트워크 트래픽 라우팅
└── Container Runtime	# 실제 컨테이너 실행 (containerd 등)

- 사용자가 `kubectl` 명령을 치면 → `kube-apiserver` 가 받아서 → `etcd` 에 기록하고 → 각 컴포넌트들이 그 변화를 감지해서 동작.
- 모든 통신은 **kube-apiserver**를 거침 (다른 컴포넌트끼리 직접 통신 안 함).

2. Docker vs. ContainerD

- 과거엔 쿠버네티스가 Docker를 직접 사용했지만, 현재는 **containerd** 같은 CRI(Container Runtime Interface) 표준을 따르는 런타임을 사용.
- Docker는 너무 많은 기능(빌드, CLI, 네트워크 등)을 포함하고 있어서 쿠버네티스 입장에선 무거움 → CRI 표준화로 더 가벼운 런타임만 사용하게 변경됨.
- **Dockershim 제거** (1.24부터): Docker 자체는 CRI를 직접 구현 안 해서, 쿠버네티스가 Dockershim이라는 중간 어댑터를 통해 Docker를 써왔는데 이게 제거됨 → 이제 Docker 이미지를 직접 쓸 수는 없고 (containerd가 OCI 표준 이미지를 사용하므로 Docker로 빌드한 이미지 자체는 호환됨), Docker 데몬을 쿠버네티스가 직접 제어하지 않음.
- 현재 주로 쓰는 런타임: **containerd**, CRI-O

현재 클러스터가 어떤 런타임 쓰는지 확인

```
kubectl get nodes -o wide
```

CONTAINER-RUNTIME 컬럼에 containerd://... 식으로 표시됨

```
crictl ps          # docker ps 대신 사용
```

```
crictl images      # docker images 대신 사용
```

3. ETCD for Beginners

etcd는 **key-value** 형태로 데이터를 저장하는 분산 저장소.

- 단순 구조: `key=value` 형태로 저장/조회
- 쿠버네티스가 등장하기 전부터 있던 독립적인 오픈소스 프로젝트
- 분산 환경에서 여러 노드 간 데이터 일관성을 보장하는 **Raft 합의 알고리즘** 사용

```
# etcd 기본 명령어 (etcdctl)
etcdctl put key1 value1
etcdctl get key1
```

4. ETCD in Kubernetes

쿠버네티스의 모든 상태 정보(클러스터 정보, Pod, Service, Secret 등)는 etcd에 저장됨.

- etcd가 곧 쿠버네티스의 "진짜 데이터베이스" — etcd가 망가지면 클러스터 상태 정보를 통째로 잃음
- `kubectl get` 으로 조회하는 모든 리소스는 사실 etcd에서 읽어오는 것
- 클러스터 구성 방식에 따라 두 가지:
 - **Stacked etcd**: master 노드 안에 etcd가 같이 떠 있음 (kubeadm 기본 방식)
 - **External etcd**: etcd를 별도 서버 군으로 분리 운영 (더 안정적이지만 복잡)
- 고가용성(HA) 클러스터에서는 etcd도 여러 대(보통 홀수, 3/5대)로 클러스터링해서 운영

5. ETCD - Commands

```
# etcd 버전 확인
etcdctl version

# key-value 저장/조회
etcdctl put key1 value1
etcdctl get key1
etcdctl get / --prefix --keys-only  # 모든 key 목록 보기

# 쿠버네티스 etcd는 TLS 인증서 필요 (인증서 경로 지정 필수)
etcdctl --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  get / --prefix --keys-only
```

```
# API 버전 지정 (v3 사용 권장)
export ETCCTL_API=3
```

백업/복구도 중요 (CKA 단골 문제):

```
# 백업
ETCCTL_API=3 etcdctl snapshot save snapshot.db \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key

# 복구
ETCCTL_API=3 etcdctl snapshot restore snapshot.db \
  --data-dir=/var/lib/etcd-from-backup
```

6. Kube-API Server

쿠버네티스의 모든 요청이 거쳐가는 중앙 관문. `kubectl` 명령도 결국 API 서버에 HTTP 요청을 보내는 것.

요청 처리 흐름

```
kubectl 명령
→ kube-apiserver 가 요청 받음
→ Authentication (누구나)
→ Authorization (권한 있냐)
→ Admission Controller (요청 내용 검증/수정)
→ etcd 에 저장
→ 변경사항을 다른 컴포넌트들에게 전파 (scheduler, kubelet 등)
```

- static pod로 떠 있음 (`/etc/kubernetes/manifests/kube-apiserver.yaml`)
- 옵션 확인:

```
kubectl describe pod kube-apiserver-master -n kube-system
cat /etc/kubernetes/manifests/kube-apiserver.yaml
```

7. Kube Controller Manager

여러 컨트롤러들을 하나로 묶어서 관리하는 프로세스. 클러스터를 "원하는 상태(desired state)"로 유지하는 역할.

주요 컨트롤러들

Node Controller → 노드 상태 감시, 다운되면 처리
Replication Controller → Pod 개수 유지
Deployment Controller → 배포 관리
Service Controller → Service/Endpoint 관리

- 각 컨트롤러는 지속적으로 etcd 상태를 감시(watch)하면서 "현재 상태 vs 원하는 상태"가 다르면 맞춰 주는 루프를 돌림
- static pod로 떠 있음:

```
cat /etc/kubernetes/manifests/kube-controller-manager.yaml
```

8. Kube Scheduler

새로 생성된 Pod를 어느 노드에 배치할지 결정하는 컴포넌트.

동작 2단계

1. Filtering: 조건에 안 맞는 노드 제외 (자원 부족, taint 등)
2. Scoring: 남은 노드들 중 가장 적합한 노드에 우선순위 매겨서 선택

- 스케줄러는 Pod를 직접 실행하지 않음 — nodeName 필드만 채워줌, 실제 실행은 해당 노드의 kubelet이 함
- static pod로 떠 있음:

```
cat /etc/kubernetes/manifests/kube-scheduler.yaml
```

(Multiple Schedulers, Scheduler Profiles는 Scheduling 섹션에서 이미 정리함)

9. Kubelet

각 워커 노드에 떠서 Pod를 실제로 실행/관리하는 에이전트.

kube-apiserver가 "이 노드에 이 Pod 떠야 해" 라고 지시
→ kubelet이 받아서
→ Container Runtime(containerd)에게 컨테이너 실행시킴
→ Pod 상태를 주기적으로 kube-apiserver에 보고

- kubelet은 **마스터 노드 컴포넌트가 아니라 워커 노드 컴포넌트** — kubeadm으로 클러스터 구성 시 kubelet은 수동 설치 필요 (kubeadm이 자동 설치 안 해줌)
- Static Pod를 직접 관리하는 주체이기도 함 (Scheduling 섹션 11번 참고)

```
systemctl status kubelet  
journalctl -u kubelet # 로그 확인 (트러블슈팅 단골)
```

10. Kube Proxy

Service ↔ Pod 사이의 **네트워크 트래픽 라우팅** 담당 (Scheduling 섹션에서 이미 다룸).

- 모든 노드에 DaemonSet으로 배포됨
- Pod의 IP가 계속 바뀌어도 Service 이름으로 안정적으로 통신 가능하게 해주는 핵심 컴포넌트
- iptables 규칙(또는 IPVS)을 이용해서 Service IP로 들어온 트래픽을 실제 Pod IP로 전달

```
kubectl get pods -n kube-system | grep kube-proxy
```

11. Pods

쿠버네티스에서 **배포 가능한 가장 작은 단위**. 컨테이너 하나 또는 여러 개를 감싸는 래퍼.

- 보통 Pod 하나 = 컨테이너 하나가 일반적 (사이드카 패턴일 때만 여러 개)
- 같은 Pod 안의 컨테이너들은 네트워크/스토리지를 공유 (같은 IP, localhost로 통신 가능)

Imperative 방식으로 빠르게 생성

```
kubectl run nginx --image=nginx  
kubectl get pods  
kubectl describe pod nginx  
kubectl delete pod nginx
```

12. Pods with YAML

선언적(Declarative) 방식으로 Pod 정의.

```
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  labels:
    app: myapp
spec:
  containers:
  - name: nginx-container
    image: nginx
```

4대 필수 필드

apiVersion	- 어떤 API 버전 쓸지 (Pod는 v1)
kind	- 무슨 오브젝트인지 (Pod, Deployment 등)
metadata	- 이름, 라벨 등 식별 정보
spec	- 실제 원하는 상태 (컨테이너 목록 등)

```
kubectl create -f pod-definition.yaml
kubectl apply -f pod-definition.yaml # 차이는 20번에서
```

13. Recap - ReplicaSets

(Scheduling 섹션 Labels and Selectors에서 일부 다룬 내용 + 추가 정리)

ReplicaSet은 **지정한 개수만큼 Pod를 항상 유지**해주는 컨트롤러.

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: myapp-replicaset
  labels:
    app: myapp
spec:
  replicas: 3
  selector:
```

```

matchLabels:
  tier: front-end
template:
  metadata:
    labels:
      tier: front-end
  spec:
    containers:
      - name: nginx
        image: nginx

```

ReplicationController(구) vs ReplicaSet(신)

ReplicationController → 구버전 (apiVersion: v1)
 ReplicaSet → 신버전 (apiVersion: apps/v1), selector 필수

핵심 규칙 재확인: selector.matchLabels 와 template.metadata.labels 반드시 일치.

```

kubectl scale --replicas=6 -f replicaset-definition.yaml
kubectl scale rs myapp-replicaset --replicas=6

```

14. Deployments

ReplicaSet을 한 단계 더 감싸서 **무중단 배포, 롤백, 버전 관리**까지 지원하는 오브젝트.

```

Deployment
  ↓ 관리
ReplicaSet
  ↓ 관리
Pod Pod Pod

```

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: myapp

```

```

template:
  metadata:
    labels:
      app: myapp
  spec:
    containers:
      - name: nginx
        image: nginx

```

핵심 기능

Rolling Update → Pod를 하나씩 순차 교체 (서비스 중단 없음)
 Rollback → 문제 생기면 이전 버전으로 되돌림
 Revision 관리 → 배포 이력 기록

```

kubectl create deployment myapp --image=nginx
kubectl get deployments
kubectl rollout status deployment/myapp
kubectl rollout history deployment/myapp
kubectl rollout undo deployment/myapp          # 직전 버전으로 롤백
kubectl rollout undo deployment/myapp --to-revision=2
kubectl set image deployment/myapp nginx=nginx:1.18

```

15. Services

Pod에 고정된 접근 경로(IP/DNS)를 제공하는 오브젝트. Pod는 계속 죽고 다시 생기면서 IP가 바뀌지만 Service는 고정된 진입점 역할.

```

apiVersion: v1
kind: Service
metadata:
  name: myapp-service
spec:
  type: ClusterIP
  selector:
    app: myapp          # 이 라벨 가진 Pod로 트래픽 전달
  ports:
    - port: 80          # Service 자체 포트
      targetPort: 8080  # Pod 컨테이너 포트

```


Service Type 3가지

ClusterIP → 클러스터 내부에서만 접근 가능 (기본값)
NodePort → 외부에서 노드 IP:포트로 접근 가능
LoadBalancer → 클라우드 로드밸런서 자동 생성 (외부 트래픽 분산)

```
kubectl expose pod nginx --port=80 --target-port=8080 --name=nginx-service
kubectl get svc
kubectl describe svc myapp-service
```

16. Services - Cluster IP

기본 Service 타입. 클러스터 내부에서만 접근 가능한 가상 IP.

- 여러 Pod(같은 라벨)들 앞에 "내부용 로드밸런서" 역할
- 마이크로서비스 간 통신(예: 백엔드가 DB Pod한테 요청)에 주로 사용

```
apiVersion: v1
kind: Service
metadata:
  name: backend-service
spec:
  type: ClusterIP # 생략 시 기본값
  selector:
    app: backend
  ports:
    - port: 80
      targetPort: 8080
```

```
kubectl create service clusterip myapp --tcp=80:8080
```

내부 DNS 이름으로도 접근 가능: 서비스이름.네임스페이스.svc.cluster.local

17. Services - LoadBalancer

외부 트래픽을 클러스터 안으로 받기 위한 Service 타입. 주로 클라우드 환경(AWS, GCP, Azure)에서 사용.

```
apiVersion: v1
kind: Service
```

```
metadata:
  name: myapp-service
spec:
  type: LoadBalancer
  selector:
    app: myapp
  ports:
  - port: 80
    targetPort: 8080
```

- 클라우드 제공자의 로드밸런서를 자동으로 프로비저닝 → 외부 IP 부여
- 온프레미스/로컬 환경(minikube 등)에서는 자동 생성 안 됨 → <pending> 상태로 멈춰 있을 수 있음 (이 경우 NodePort로 우회 테스트하거나 MetalLB 같은 추가 도구 필요)

```
kubectl get svc myapp-service
# EXTERNAL-IP 컬럼이 <pending> 이면 클라우드 LB 프로비저닝 대기 중
```

NodePort와 차이: NodePort는 모든 노드의 특정 포트(30000-32767)로 직접 노출, LoadBalancer는 그 위에 외부 LB까지 자동으로 붙여줌.

18. Namespaces

같은 클러스터 안에서 리소스를 논리적으로 격리하는 단위.

```
kubectl get namespaces
# 기본 제공: default, kube-system, kube-public, kube-node-lease
```

- kube-system → 쿠버네티스 시스템 컴포넌트들 (kube-proxy, coredns 등)
- default → namespace 안 적으면 여기로 들어감
- 같은 namespace 안에서는 리소스 이름만으로 서로 통신 가능 (서비스이름), 다른 namespace면 서비스이름.네임스페이스 필요

```
kubectl create namespace dev
kubectl get pods -n dev
kubectl get pods --all-namespaces      # 모든 namespace 통합 조회

# yaml에 네임스페이스 지정
metadata:
  name: myapp
  namespace: dev
```

Resource Quota: namespace별로 자원 사용량 제한 가능

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: compute-quota
  namespace: dev
spec:
  hard:
    pods: "10"
    requests.cpu: "4"
    requests.memory: 5Gi
```

19. Imperative vs. Declarative

Imperative (명령형): "이렇게 해라" 하고 직접 명령어로 지시

```
kubectl run nginx --image=nginx
kubectl create deployment myapp --image=nginx
kubectl expose pod nginx --port=80
```

Declarative (선언형): "이런 상태가 되었으면 좋겠다" 를 yaml로 정의하고 적용

```
kubectl apply -f deployment.yaml
```

	Imperative	Declarative
방식	명령어로 직접 지시	yaml로 원하는 상태 선언
속도	빠름 (시험에서 유리)	느림 (yaml 작성 필요)
버전 관리	어려움	git 등으로 관리 용이
실무	빠른 테스트용	운영 환경 표준

CKA 시험 전략: 시간 제한이 있어서 **Imperative 명령어 + --dry-run** 으로 **yaml 빠르게 생성 후 수정하는 하이드리드 방식**이 제일 빠름.

```
kubectl run nginx --image=nginx --dry-run=client -o yaml > pod.yaml
```

20. Certification Tips - Imperative Commands with Kubectl

시험에서 자주 쓰는 단축 생성 명령어 모음.

```
# Pod 생성
kubectl run nginx --image=nginx

# Deployment 생성
kubectl create deployment nginx --image=nginx --replicas=3

# Deployment 생성 후 yaml로 뽑아서 수정
kubectl create deployment nginx --image=nginx --dry-run=client -o yaml > deploy.yaml

# Service 생성
kubectl expose pod nginx --port=80 --target-port=8080
kubectl create service clusterip myapp --tcp=80:8080
kubectl create service nodeport myapp --tcp=80:30080

# Namespace 생성
kubectl create namespace dev

# ConfigMap / Secret 생성
kubectl create configmap my-config --from-literal=key1=value1
kubectl create secret generic my-secret --from-literal=password=1234

# 빠르게 yaml 뽑고 수정해서 적용하는 패턴 (CKA 핵심 전략)
kubectl create deployment nginx --image=nginx --dry-run=client -o yaml > nginx.yaml
vi nginx.yaml # 필요한 부분만 수정 (replicas, resources 등)
kubectl apply -f nginx.yaml
```

--dry-run=client -o yaml 은 실제로 생성하지 않고 yaml만 뽑아주는 옵션. 이 조합이 CKA 시험에서 시간을 가장 많이 절약해줌.

21. Kubectl Explain Commands

오픈북 시험이지만 공식 문서보다 더 빠르게 필드 구조를 확인할 수 있는 내장 명령어.

```
kubectl explain pod
kubectl explain pod.spec
kubectl explain pod.spec.containers
```

```
kubectl explain deployment.spec.strategy
```

```
# 모든 하위 필드까지 한 번에 보기
```

```
kubectl explain pod --recursive
```

- yaml 작성 중 "이 필드 이름이 뭐였더라" 할 때 브라우저로 문서 찾아보는 것보다 훨씬 빠름
- 필드별 타입(string, integer, object 등)과 필수 여부도 같이 알려줌

```
kubectl explain replicaset.spec.selector
```

```
# FIELDS: matchLabels <map[string]string>, matchExpressions <[]Object> 등 표시
```

22. Kubectl Apply Command

`kubectl apply` 는 선언적 방식의 핵심 명령어. yaml 파일과 실제 클러스터 상태를 비교해서 **차이나는 부분만 업데이트**.

3가지 비교 대상

1. Local 파일 (지금 적용하려는 yaml)
2. Last Applied Configuration (직전에 apply했던 내용, annotation에 저장됨)
3. Live Object Configuration (현재 클러스터의 실제 상태, etcd에 저장된 값)

```
kubectl apply -f pod.yaml
```

- 동작 원리: local 파일과 last-applied-config를 비교해서 **추가/삭제/변경된 필드**를 계산 → 그 결과를 live object에 patch로 반영
- 그래서 yaml에서 필드를 지워도 apply 시 클러스터에서도 그 필드가 제거됨 (last-applied-config 덕분에 추적 가능)

create vs apply 차이

```
kubectl create -f pod.yaml → 이미 존재하면 에러남 (한 번만 생성 가능)
```

```
kubectl apply -f pod.yaml → 없으면 생성, 있으면 차이만 업데이트 (반복 실행 가능)
```

```
# 현재 적용된 last-applied-configuration 확인
```

```
kubectl get pod nginx -o yaml
```

```
# metadata.annotations.kubectl.kubernetes.io/last-applied-configuration 에서 확인 가능
```

CKA에서는 보통 빠른 작업엔 `create`, 반복 수정이 필요한 yaml 기반 작업엔 `apply` 를 사용.

CKA 시험 팁 요약

- 시험은 **오픈북** — kubernetes.io/docs 검색 가능. 명령어 외우기보다 구조 이해 + `kubectl explain` 적극 활용.
- 시간이 부족하므로 **Imperative 명령 + `--dry-run=client -o yaml`** 조합으로 yaml 빠르게 생성 후 수정하는 습관 필수.
- 트러블슈팅은 항상 컴포넌트 단위로 점검: `kubectl get pods -n kube-system`, `kubectl describe`, `journalctl -u kubelet`, static pod manifest 파일(`/etc/kubernetes/manifests/`) 확인.
- etcd 백업/복구는 단골 실습 문제이므로 명령어 흐름(snapshot save → restore → 클러스터 재구성)을 꼭 익혀둘 것.